Shell

Abhilash Jindal

Agenda

- Understand how shell works (xv6 book Ch0: processes, pipes)
  - Process APIs: fork, wait, exec (OSTEP Ch5)
  - IO redirection, pipes
- Implementations:
  - Fork (xv6 Ch1), copy-on-write (OSTEP Ch23)
  - Exec, loading ELF binaries and running (xv6 Ch2)
  - Pipes, wait, exit (xv6 Ch5)

Shell

- Fork, exec, wait, IO redirection, pipes:
  - ls
  - echo hello > world.txt
  - wc < world.txt
  - echo hello | wc
  - yes OS. In new terminal, ps aux | grep yes; kill <pid>

Fork Semantics

- p1.c: Child gets rc=0 and parents gets child's PID in rc
- fork-cow.c: Address space is copied at the time of fork. Writes of child and parent are not seen to each other
  - Example: each browser tab can be a separate process forked by the main browser
  - Redis forks a backup process to checkpoint in-memory database to disk
- fork-fd.c: OS is maintaining an open file table for each process. Open file table is just an array from file descriptor (integer) to a file struct (containing offsets). Open file table is duplicated at time of fork. When one process writes, the offset is incremented.
  - Example: Browser tabs can write to a common log file without stepping over one another
- dup.c: Duplicate file descriptors can be created within a process. dup copies file descriptors along with offsets. Compare output with nodup.c

Other process APIs and IO redirection

- p2.c: Shell wants to be able to wait for the child process to finish. wait system call does that

- p3.c: Shell does not want to run just copies of itself. It wants to execute other program binaries. exec prepares a new address space where it loads the program binary stored in the ELF format. Exec gives control to the ELF->entry

- p4.c: In the gap between fork and exec, shell can open and close files. By default processes have stdin file (fd=0), stdout file (fd=1), and stderr file (fd=2). Here we close stdout and open “p4.output”. Child will just write to file descriptor 1 which now points to “p4.output”

- pipe.c: pipe returns two file descriptors: a read end and a write end. We close STDOUT and dup the write end. In parent, we close STDIN and dup the read end.

fork in xv6 (proc.c)

- Allocate process: allocate a kernel stack, etc
- Copy memory (copyuvm): copy the page table and all the process’ pages
- Copy trap frame, set return value as 0 in the child process
- Duplicate file descriptors (shared file offset)
- Mark child as runnable

fork in xv6 (proc.c)

```
int fork(void) {
  int i, pid;
  struct proc *np;
  struct proc *curproc = myproc();
  np = allocproc()

  np->pgdir = copyuvm(curproc->pgdir, curproc->sz)
  np->sz = curproc->sz;
  np->parent = curproc;

  *np->tf = *curproc->tf;
  // Clear %eax so that fork returns 0 in the child.
  np->tf->eax = 0;
```

Allocate a new process, its kstack, pgdir etc

Copy the page table and all the process' pages

Copy trap frame. Set return value as zero in child process

Duplicate all open file descriptors (shared file offset)

```
  for(i = 0; i < NOFILE; i++)
    if(curproc->ofile[i])
      np->ofile[i] = filedup(curproc->ofile[i]);
  np->cwd = idup(curproc->cwd);

  safestrcpy(np->name, curproc->name, ..));
  pid = np->pid;
  acquire(&ptable.lock);
  np->state = RUNNABLE;
  release(&ptable.lock);
  return pid;
}
```

Mark new process as runnable

Faster forks

- Addresses may be large => very slow fork
- Use copy-on-write

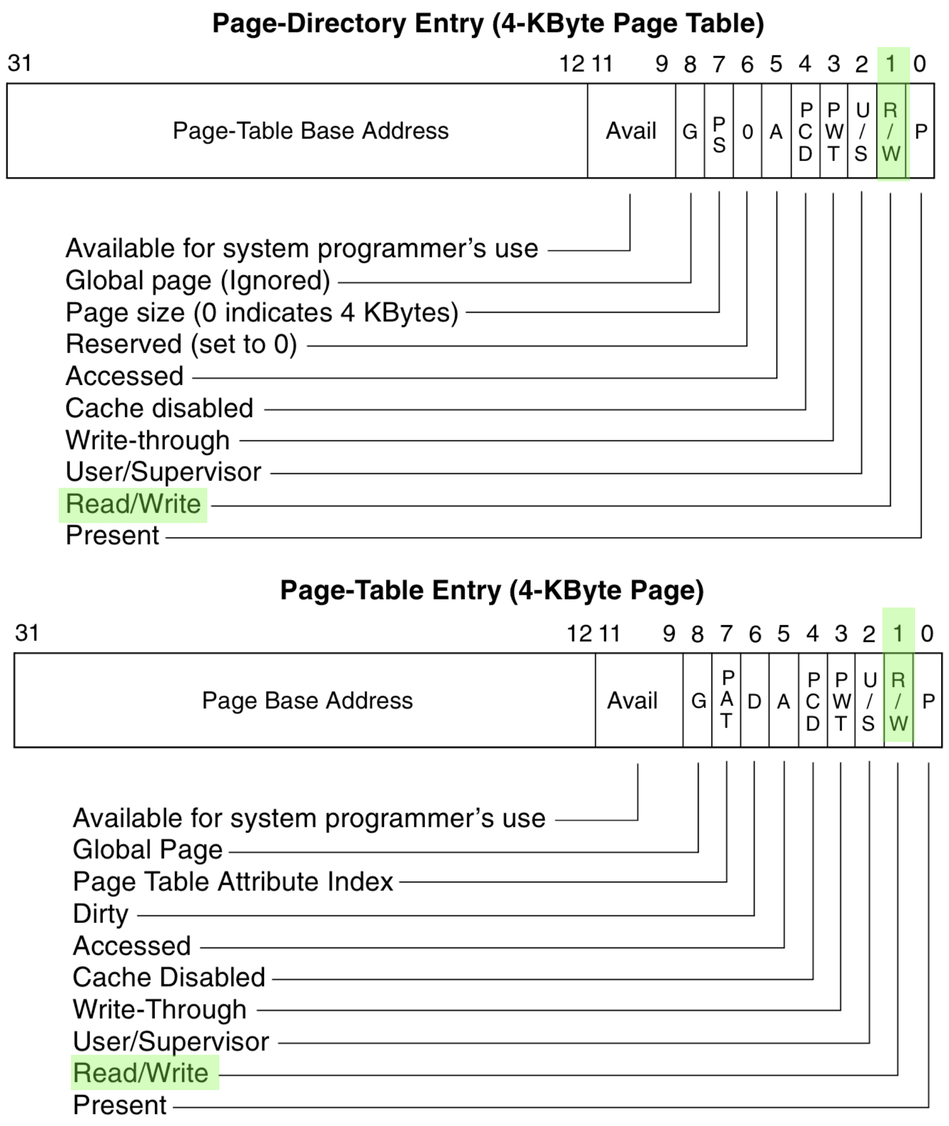
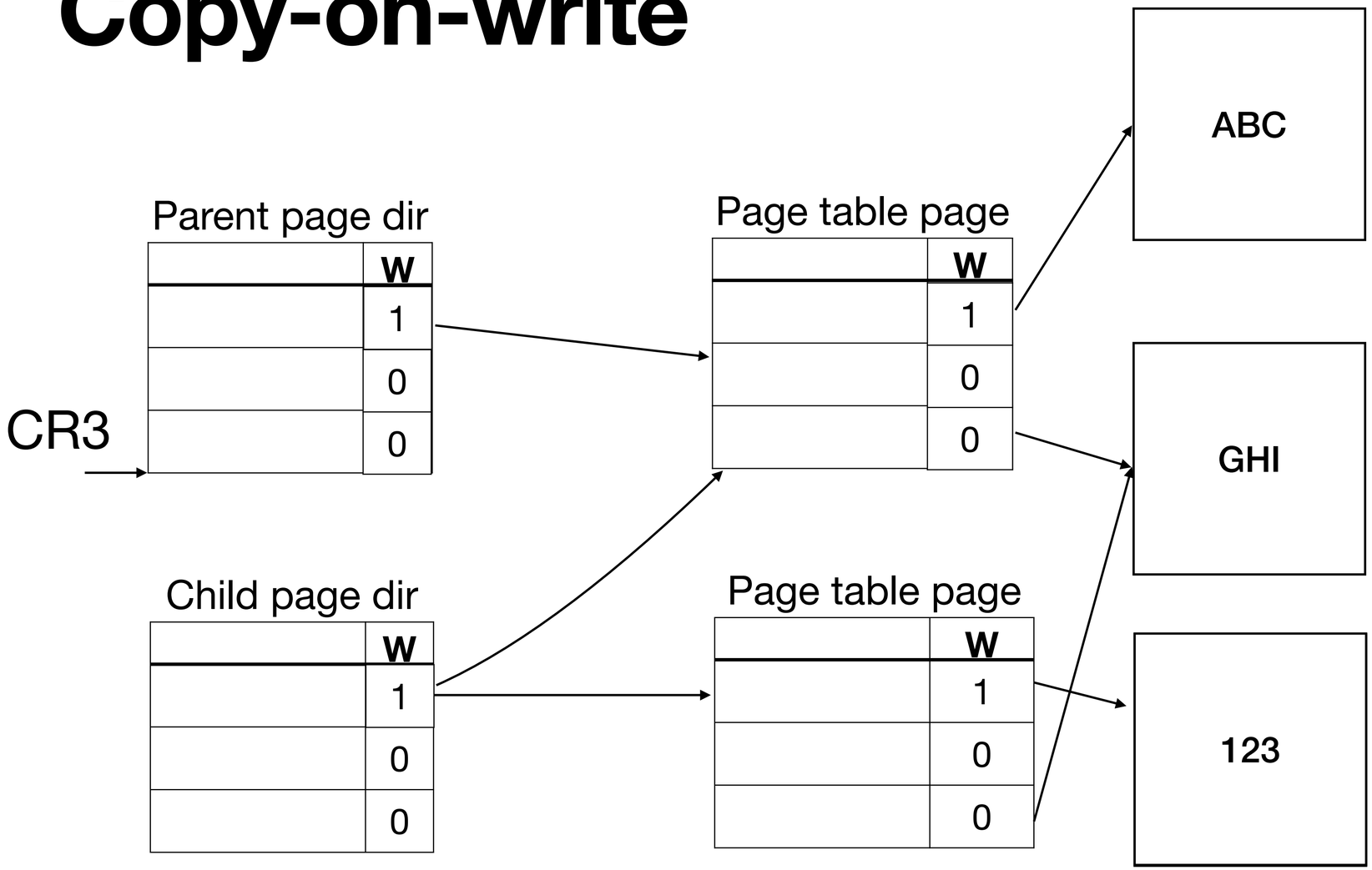


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Copy-on-write



Parent forks

Child process overwrites “ABC” with “123”

exec in xv6 (exec.c)

- Allocate a new page table
- Read ELF file to load program from disk to memory
  - Call allocuvm to allocate space and create entries in the page table
  - loaduvm to copy ELF segments
- Add two pages above text and data for stack and guard page
- Remove user flag from the guard page so that if stack is overflowing into data, kernel gets a page fault
- Copy arguments on the stack, mark eip as elf.entry, switch to the new page table, and return (which will return from trap)

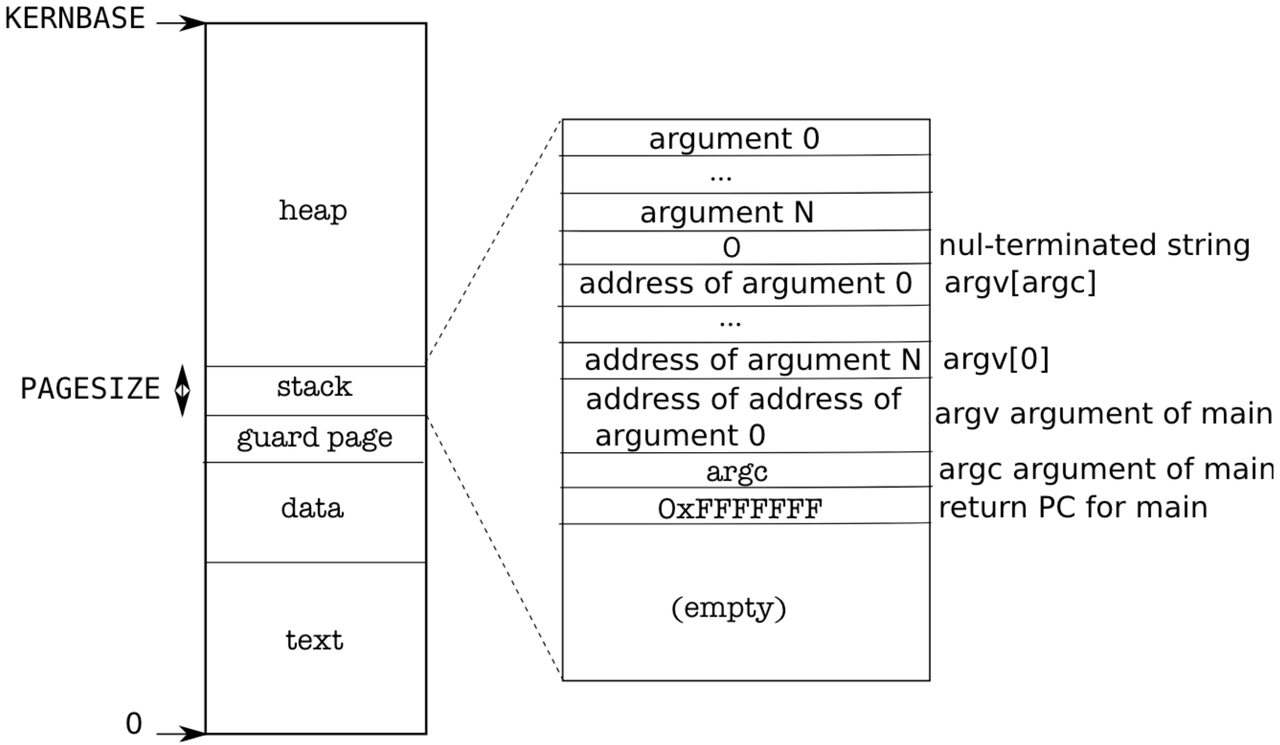


Figure 2-3. Memory layout of a user process with its initial stack.

exec in xv6 (exec.c)

```
int exec(char *path, char **argv){
  ip = namei(path);
  readi(ip, (char*)&elf, 0, sizeof(elf));

  pgdir = setupkvm()
  sz = 0;

  for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
    readi(ip, (char*)&ph, off, sizeof(ph);
    sz = allocuvm(pgdir, sz, ph.vaddr + ph.memsz);
    loaduvm(pgdir, (char*)ph.vaddr, ip, ph.off, ph.filesz);
  }
  sz = PGROUNDUP(sz);

  if((sz = allocuvm(pgdir, sz, sz + 2*PGSIZE)) == 0)
    goto bad;
  clearpteu(pgdir, (char*)(sz - 2*PGSIZE));
```

Read ELF file header from disk

Setup a new page table

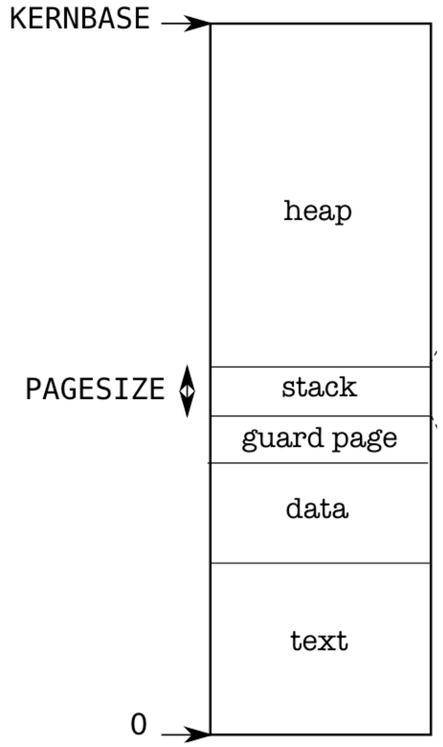
Read ELF sections one-by-one

Allocate space: create entries in the page table

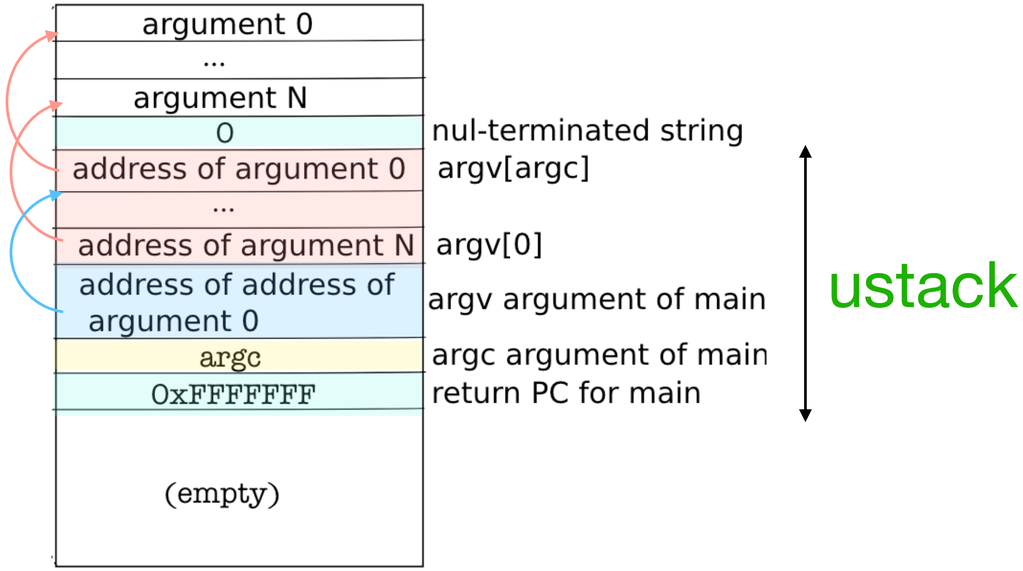
Copy ELF segment into the new address space

Add two pages above the loaded text+data: stack and guard

Unset user bit from the lower (guard) page



exec in xv6 (exec.c)




```
sp = sz;
// Push argument strings, prepare rest of stack in ustack.
for(argc = 0; argv[argc]; argc++) {
  sp = (sp - (strlen(argv[argc]) + 1)) & ~3;
  copyout(pgdir, sp, argv[argc], strlen(argv[argc]) + 1);
  ustack[3+argc] = sp;
}
ustack[3+argc] = 0;
ustack[0] = 0xffffffff;  // fake return PC
ustack[1] = argc;
ustack[2] = sp - (argc+1)*4;  // argv pointer

sp -= (3+argc+1) * 4;
copyout(pgdir, sp, ustack, (3+argc+1)*4);

// Commit to the user image.
oldpgdir = curproc->pgdir;
curproc->pgdir = pgdir;
curproc->sz = sz;
curproc->tf->eip = elf.entry;  // main
curproc->tf->esp = sp;
switchuvm(curproc);
freevm(oldpgdir);
```

Set esp to top of stack and eip to ELF entry in trap frame

Change CR3 to point to the new address space (OS is still mapped identically)

Free up the old address space

Executing initial process in xv6 (initcode.S, init.c)

- initcode.S just calls exec with “/init”
- init.c opens the default stdin, stdout, and stderr files. It calls fork and starts shell process. init process itself stays in an infinite loop waiting for child processes to exit
- sh.c is in an infinite loop waiting for user input. It forks+execs process(es) and waits for them to finish before again waiting for user input

exit in xv6 (proc.c)

- When a process ends (see cat.c for example), it calls the exit system call
- exit closes all the open file descriptors, wake up the parent who might be waiting for the child to exit
- If parent exits before the child, it makes initproc as the parent. If child has already exited, it wakes up initproc for cleanup
- Note that wakeup only marks the process as runnable but does not call the scheduler
- Finally, the process marks itself as zombie and calls the scheduler

exit in xv6 (proc.c)

```
void exit(void) {
  struct proc *curproc = myproc();

  // Close all open files.
  for(fd = 0; fd < NOFILE; fd++){
    if(curproc->ofile[fd]){
      fileclose(curproc->ofile[fd]);
      curproc->ofile[fd] = 0;
    }
  }
  acquire(&ptable.lock);
  // Parent might be sleeping in wait().
  wakeup1(curproc->parent);

  // Pass abandoned children to init.
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == curproc){
      p->parent = initproc;
      if(p->state == ZOMBIE)
        wakeup1(initproc);
    }
  }

  // Jump into the scheduler, never to return.
  curproc->state = ZOMBIE;
  sched();
  panic("zombie exit");
}
```

wait in xv6 (proc.c)

- wait cleans up the process states. It returns after cleaning up a child process
- It loops over process table to find out a zombie child process and cleans up their kernel stack, page table, and finally mark the process slot as UNUSED
- If it has running children (not ZOMBIE), it goes into sleep to be woken up whenever a child process exits

wait in xv6 (proc.c)

```
int wait(void) {
  struct proc *p;
  int havekids, pid;
  struct proc *curproc = myproc();
  acquire(&ptable.lock);
  for(;;){
    havekids = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->parent != curproc)
        continue;
      havekids = 1;
      if(p->state == ZOMBIE){
        [redacted]
        return pid;
      }
    }
    if(!havekids || curproc->killed){
      release(&ptable.lock);
      return -1;
    }
    sleep(curproc, &ptable.lock);
  }
}
```

Found a process that has exited and waiting to be cleaned up

Sleep on my PCB. When a child exits, it will wake me up!

```
// Clean up
pid = p->pid;
kfree(p->kstack);
p->kstack = 0;
freevm(p->pgdir);
p->pid = 0;
p->parent = 0;
p->name[0] = 0;
p->killed = 0;
p->state = UNUSED;
release(&ptable.lock);
```

Clean up child process' kstack, and address space

Marks process state as UNUSED. A new process can reuse this PCB now

kill in xv6 (kill.c, proc.c)

- kill(pid): we do not want to immediately kill a process. It may be in middle of something (like writing disk blocks)
- kill(pid) only marks p->killed=1 and marks it as runnable if it was sleeping
- After waking up, we usually check if the process is already killed and gracefully clean up (see console.c, pipe.c, sysproc.c)
- When entering the OS or when returning to process, if process is killed, we call exit (see trap.c). These are good places to exit since kernel was not in the middle of servicing requests from the process.

Exit when come into or when returning from OS

```
void trap(struct trapframe *tf) {
  if(tf->trapno == T_SYSCALL){
    if(myproc()->killed) exit();
    syscall();
    if(myproc()->killed) exit();
    return;
  }
  switch(tf->trapno){
  ...
  default:
    if(myproc() && (tf->cs&3) == 3) myproc()->killed = 1;
  }

  if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER) exit();
  if(myproc() && myproc()->state == RUNNING &&
      tf->trapno == T_IRQ0+IRQ_TIMER)
    yield();
  if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER) exit();
}
```

Exit before/after handling syscall

If there was an unexpected trap, just kill the process

Exit before/after yielding

pipe in xv6 (pipe.c)

- See “case PIPE” in sh.c. It calls pipe sys call and passes it an integer pointer pointing to two integers
- sys_pipe (sysfile.c) calls pipealloc which returns two file descriptors: a read end of the pipe and a write end of the pipe and adds the descriptors to open file table of the current process (shell) and writes the two file descriptors in the integers pointed to by the argument
- Pipe behaves just like files: fileread/filewrite/fileclose is forwarded to piperead/pipewrite/pipeclose (see file.c)
- sh.c forks the reader (writer) process, closes its stdin (stdout), and duplicates read side (write side) of the pipe and closes p[0] and p[1] descriptors in all processes: reader/writer/shell.
- pipealloc (in pipe.c) allocates one page to hold the pipe struct which has 512 bytes of “data”. Data will be transferred between processes via this “data” (i.e, without going through disk)
- Try to understand rest of pipe.c. We will look at it when we talk about locks!